

A) A distance matrix for 5 City is Provided below the branch and bound algorithm is about to start TSP. What will be the lower bound of cost of the tool as per the algorithm.

Estimate the (lower bound) cost of the tool that exclude the BC Segment.

	A	B	C	D	E
A	-	70	40	72	42
B	70	-	30	32	68
C	40	30	-	36	44
D	72	32	36	-	50
E	42	68	44	50	-

i) Initial lower bound without constrain

Edge A = 2 Smallest cost of $\{70, 40, 72, 42\} = (40, 42)$

Sum of 2 Smallest cost = $40 + 42 = 82$

Edge B = 2 Smallest Cost of $\{70, 30, 32, 68\} = (30, 32)$

Sum of 2 Smallest Cost = $30 + 32 = 62$

Edge C = 2 Smallest Cost of $\{40, 30, 36, 44\} = (30, 36)$

Sum of 2 Smallest Cost = $30 + 36 = 66$

Edge D = 2 Smallest Cost of $\{72, 32, 36, 50\} = (32, 36)$

Sum of 2 Smallest Cost = $32 + 36 = 68$

Edge E = 2 Smallest Cost of $\{42, 68, 44, 50\} = (42, 44)$

Sum of 2 Smallest Cost = $42 + 44 = 86$

$$\text{Total Cost} = \frac{82 + 62 + 66 + 68 + 86}{2}$$

$$= \frac{364}{2}$$

$$= 182$$

Taking two lowest nodes = 182

So, Branch Bound algorithm will begin with 182.

ii) Lower bound next took that exclude the segment BC,

Recompute the 2 Smallest for B and C without (B, C) and (C, B)

$$\text{Edge A} = 82$$

$$\text{Edge B} = 32 + 68 = 100$$

$$\text{Edge C} = 40 + 36 = 76$$

$$\text{Edge D} = 68$$

$$\text{Edge E} = 84$$

$$\begin{aligned}\text{Total} &= \frac{82 + 100 + 76 + 68 + 86}{2} \\ &= \frac{412}{2} \\ &= 206\end{aligned}$$

Interpretation,

- Initial lower bound (No Constraints) = 182
- Lower bound for tours that exclude edge BC = 206

Because excluding BC raises the lower bound from 182 to 206, any already-found feasible tour with cost < 206 would mean all BC-excluded subtrees can be pruned; otherwise those branches might still contain a better tour.

Step-1

$$[\text{Min}(A, B); \text{min}(C, D); \text{min}(E, F); \dots \text{min}(W, X)]$$
$$= [20, 19, 40, 34, 16, 42, 46, 27, 18, 12, 15, 14]$$

Step-2

$$[\text{Max}(20, 19), \text{Max}(40, 34), \dots \text{Max}(15, 14)]$$
$$= [20, 40, 42, 46, 18, 15]$$

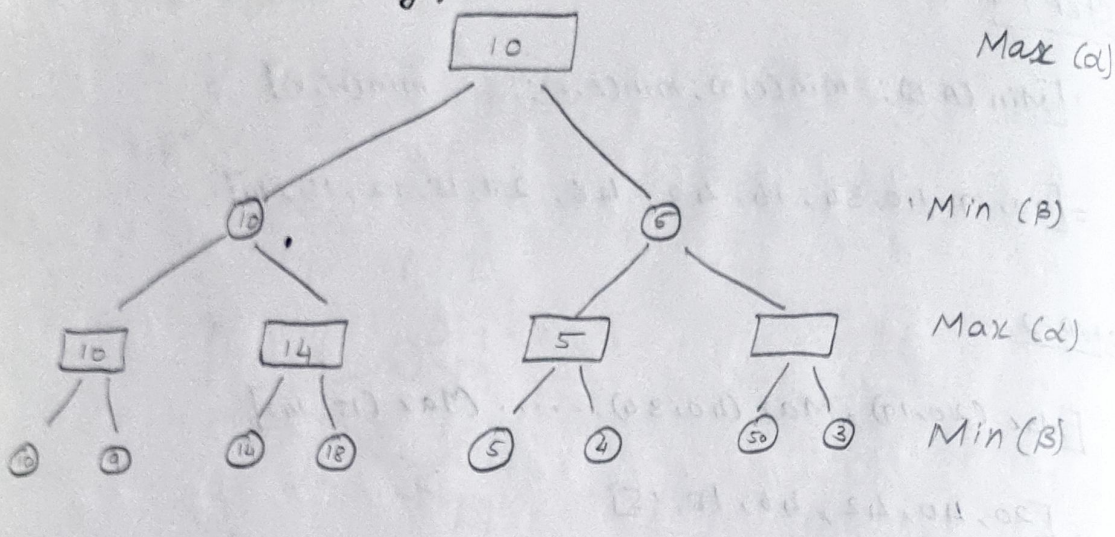
Step-3

$$[\text{Min}(20, 40), \text{min}(42, 46), \text{min}(18, 15)]$$
$$= [20, 42, 15]$$

Step-4

$$\text{max}[20, 42, 15]$$
$$= 42$$

Alpha - Beta Pruning:



Max - α

Min - β

Pruning,

Max (10, 9) $\rightarrow$ 10

Min (10, 14)

Petri Nets :

Definition of Petri Net

$$C = (P, T, I, O)$$

Places

$$P = \{P_1, P_2, P_3, \dots, P_n\}$$

Transitions

$$T = \{t_1, t_2, t_3, \dots, t_n\}$$

Input

$$I : T \rightarrow P^+ \text{ (} r = \text{number of Places)}$$

Output

$$O : T \rightarrow P^+ \text{ (} q = \text{number of Places)}$$

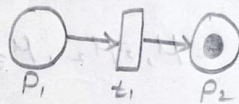
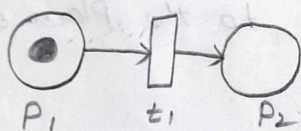
- marking μ : assignment of tokens to the Places of Petri net
 $\mu = \mu_1, \mu_2, \mu_3, \dots, \mu_n$

Applications

- Petri net is primarily used for studying the dynamic concurrent behavior of network-based systems where there is a discrete flow.
- Petri nets are applied in practice by industry, academia, and other places.

Basics of Petri Nets:

- Petri net consist two types of nodes: Places and transitions. And arc exists only from a Place to a transition or from a transition to a Place.
 - A Place may have zero or more tokens.
 - Graphically, Places, transitions, arcs and tokens are represented respectively by: Circles, bars, arrows, and dots.
- Example Petri net with two Places and one transition.
- Transition node is ready to fire if and only if there is at least one token at each of its input Places.



State transition of form $(1, 0) \rightarrow (0, 1)$

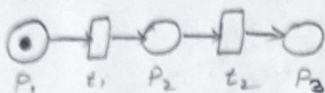
P_1 - Input Place

P_2 - Output Place

Properties of Petri Nets:

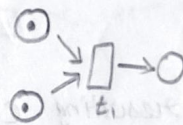
• Sequential Execution:

Transition t_2 can fire only after the firing of t_1 .
This imposes the Precedence of Constraints " t_2 after t_1 ".



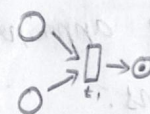
• Synchronization:

Transition t_1 will be enabled only when a token
there are at least one token at each of its input places.



• Merging:

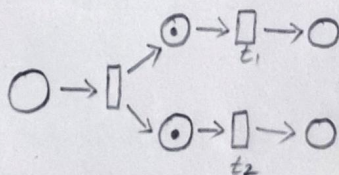
Happens when tokens from several places
arrive for service at the same transition.



• Concurrency:

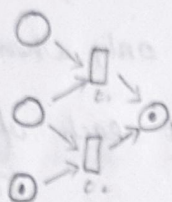
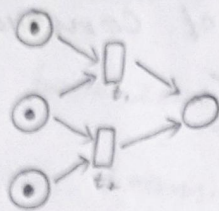
t_1 and t_2 are concurrent.

- With this property, Petri net is able to model systems of
distributed control with multiple processes executing
concurrently in time.



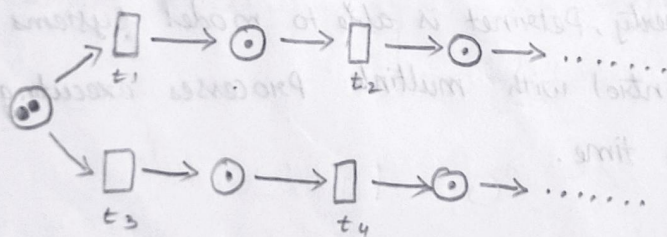
• Conflict :

- t_1 and t_2 are both ready to fire but the firing of any leads to the disabling of the other transitions.



- the resulting conflict may be resolved in a purely non-deterministic way or in a probabilistic way, by assigning appropriate probabilities to the conflicting transitions.

there is a choice of either t_1 and t_2 , or t_3 and t_4 .



Alpha - Beta Pruning:

I recorded the leaf visit order, the (α, β) window entering the leaf-Parent, and where the cutoff occurred, the important events (the points that cause pruning) are:

- At leaf index 3 ($C=19$): a β -cutoff happens - leaf 4(D) is not needed under that local node.

(Cutoff line: 3, label C; Pruned: D(4))

- At leaf index 5 ($E=40$): an α -cutoff happens - this causes two leaves (the rest of that sibling subtree) to be skipped.
- At index 9 ($I=16$): a β -cutoff occurs (no further leaves in that tiny branch to enumerate afterwards).

(Cutoff line: 9, label I.)

- At index 11 ($K=42$): an α -cutoff occurs - it prunes the next two leaves under that sibling subtree.
- At indices 15 and 17 ($O=27$ and $Q=18$): β -cutoffs occur while exploring the rightmost top subtree (they cause some subtrees further right to be skipped).

(Cutoff lines: 15 (O) and 17 (Q)).

Putting the cutoff lines together:

- Alpha-cutoffs: lines 5 and 11 (leaf labels E and K)
- Beta-cutoffs: lines 3, 9, 15, 17 (leaf labels C, I, O, Q).

Vending machine Petri net

Components:

Circle $\rightarrow$ Amount of Coin inserted

(0c, 5c, 10c, 15c, 20c)

Rectangle $\rightarrow$ Action (insert coin or take chocolate bar)

Arrows $\rightarrow$ Movement of the token (State change)

Black dot (token) $\rightarrow$ Current state

Initial state:

Token at 0 $\rightarrow$ no money inserted yet

Transaction (Action):

- Deposit 5c $\rightarrow$ add 5 cents

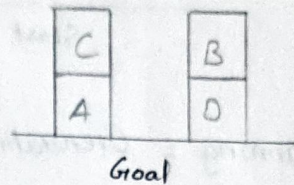
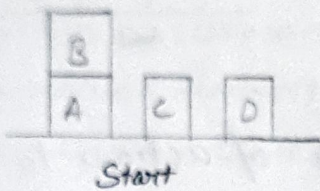
- Deposit 10c $\rightarrow$ add 10 cents

- Take 15c bar $\rightarrow$ give a chocolate bar worth 15 cents & return to 0c.

- Take 20c bar $\rightarrow$ give a chocolate bar worth 20 cents & return to 0c.

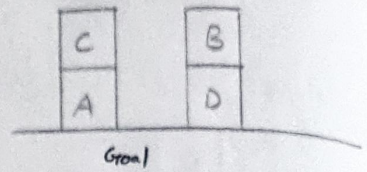
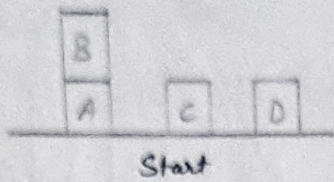
Goal Stack Planning :

① . Planning



- Problem-Solving is searching and moving through a state space.
- Planning is searching for successful paths through a state space.
- Planning is the process of computing several steps of a problem-solving procedure before executing them.
- Planning = Problem Solving in advance
- Planning is important if solutions cannot be undone
- If the universe is not predictable, then a plan can fail
⇒ Dynamic Plan revision.

②. The Blocks world



Planning = Generating a Sequence of actions to achieve the goal from the start.

Actions:

- UNSTACK (A, B)
- STACK (A, B)
- PICKUP (A)
- PUTDOWN (A)

Conditions and results:

- ON (A, B)
- ONTABLE (A)
- CLEAR (A)
- HOLDING (A)
- ARMEMPTY

Specification of actions:

- PRECONDITION: list of Predicates that must be true for an operator to be applied
- ADD: list of new predicates that an operator causes to become true
- DELETE: list of old predicates that an operator causes to become false
- Predicates not in ADD AND DELETE are unaffected.

STACK (x, y):

P: CLEAR(y) $\wedge$ HOLDING(x)

D: CLEAR(y) $\wedge$ HOLDING(x)

A: ARMEMPTY $\wedge$ ON(x, y)

UNSTACK (x, y):

P: ON(x, y) $\wedge$ CLEAR(x) $\wedge$ ARMEMPTY

D: ON(x, y) $\wedge$ ARMEMPTY

A: HOLDING(x) $\wedge$ CLEAR(y)

PICKUP (x):

P: CLEAR(x) $\wedge$ ONTABLE(x) $\wedge$ ARMEMPTY

D: ONTABLE(x) $\wedge$ ARMEMPTY

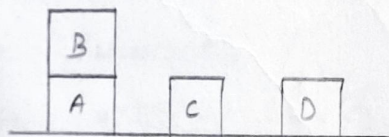
A: HOLDING(x)

PUTDOWN (x):

P: HOLDING(x)

D: HOLDING(x)

A: ONTABLE(x) $\wedge$ ARMEMPTY



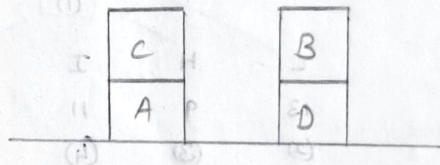
Start: ON(B, A) $\wedge$

ONTABLE(A) $\wedge$

ONTABLE(C) $\wedge$

ONTABLE(D) $\wedge$

ARMEMPTY



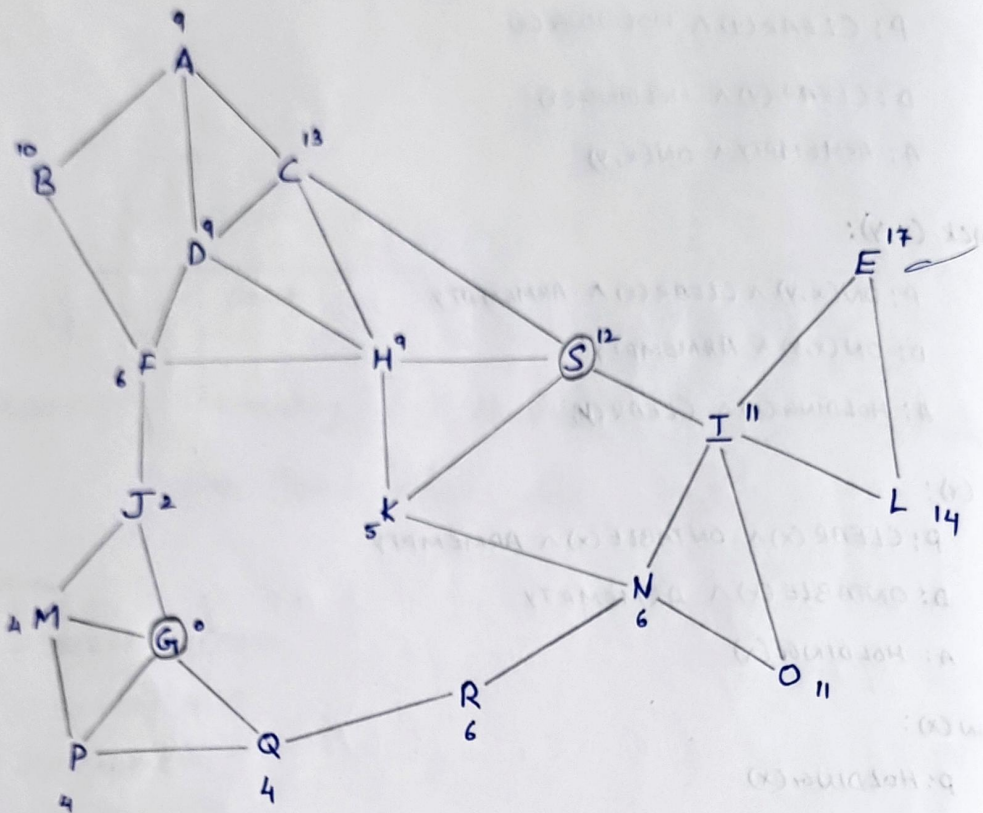
Goal: ON(C, A) $\wedge$

ON(B, D) $\wedge$

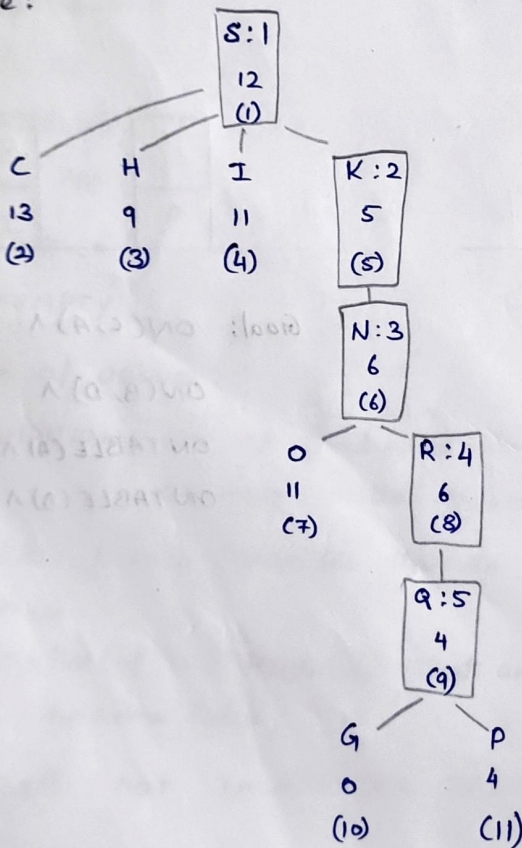
ONTABLE(A) $\wedge$

ONTABLE(D) $\wedge$

Best First Search



Search tree:



Solution:

OPEN and CLOSED carry tuples: (Node, PARENT, H-Valve)

OPEN (S, null, 12): []

CLOSED []

1.

Node

S: (S, 2, 5): (H, 11, 0): (I, 2, 1): (K, 2, 4): (C, 1, 8): []

Close

(S, null, 12)

move Gen

C: H: I: K: []

New Nodes

C: H: I: K: []

New Pairs

(C, S, 13): (H, S, 9): (I, S, 11): (K, S, 5): []

OPEN

(K, S, 5): (H, S, 9): (I, S, 11): (C, S, 13): []

CLOSED

(S, null, 12): []

2.

Node

K

Close

(K, S, 5)

move Gen

H: N: S: []

New Nodes

N: []

New Pairs

(N, K, 6): []

OPEN

(N, K, 6): (H, S, 9): (I, S, 11): (C, S, 13): []

CLOSED

(K, S, 5): (S, null, 12): []

3.

Node N
 close (N, k, 6)
 move then I : k : 0 : R : []
 new nodes 0 : R : []
 new pairs (0, N, 11) : (R, N, 6) : []

OPEN (R, N, 6) : (H, S, 9) : (I, S, 11) : (0, N, 11) : (C, S, 13) : []
 closed (N, k, 6) : (k, S, 5) : (S, null, 12) : []

4.

Node R
 close (R, N, 6)
 move then N : Q : []
 new nodes Q : []
 new pairs (Q, R, 4) : []

OPEN (Q, R, 4) : (H, S, 9) : (I, S, 11) : (0, N, 11) : (C, S, 13) : []
 closed (R, N, 6) : (N, k, 6) : (k, S, 5) : (S, null, 12) : []

5.

Node Q
 Close (Q, R, 4)
 move Gen G: P: R: []
 new nodes G: P: []
 new pairs (G, Q, 0): (P, Q, 4): []

OPEN (G, Q, 0): (P, Q, 4): (H, S, 9): (I, S, 11): (O, N, 11): (C, S, 13): []

closed (Q, R, 4): (R, N, 6): (N, K, 6): (K, S, 5): (S, null, 12): []

6.

Node G
 Goal G

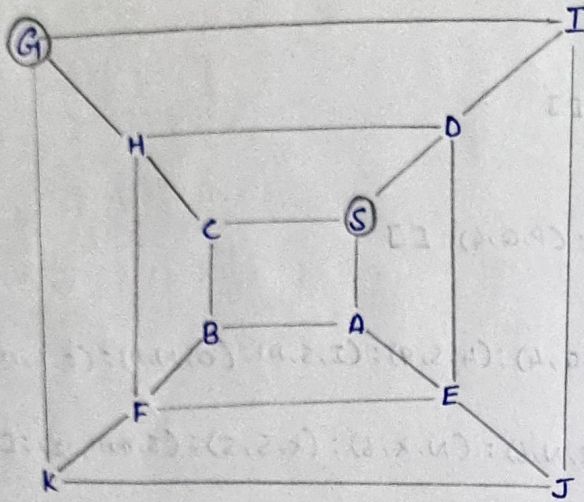
OPEN (G, Q, 0): (P, Q, 4): (H, S, 9): (I, S, 11): (O, N, 11): (C, S, 13): []

CLOSED (Q, R, 4): (R, N, 6): (N, K, 6): (K, S, 5): (S, null, 12): []

Path S: K: N: R: Q: G: []

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99

Depth First Search



Move Gen returns nodes in Alphabetical order

S → A, C, D

A → B, E, S

B → A, C, F

C → B, H, S

D → E, H, I, S

E → A, D, F, J

F → B, E, H, K

H → C, D, F, G

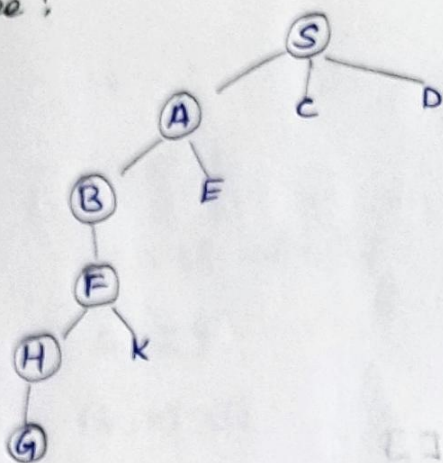
I → D, G, J

J → E, I, K

K → F, G, J

G → H, I, K

Search Tree;



Solution:

OPEN and CLOSED Carry pairs : (NODE, PARENT)

OPEN (S, null): []

CLOSED []

1. Node S

close (S, null)

move Gen A: C: D: []

new nodes A: C: D: []

new Pairs (A, S): (C, S): (D, S): []

OPEN (A, S): (C, S): (D, S): []

CLOSED (S, null): []

2.

NODE A
 close (A, S)
 Move Gen B: E: S: []
 new nodes B: E: []
 new pairs (B, A): (E, A): []
 OPEN (B, A): (E, A): (C, S): (D, S): []
 CLOSED (A, S): (S, null): []

3.

NODE B
 close (B, A)
 move Gen A: C: F: []
 new nodes F: []
 new pairs (F, B): []
 OPEN (F, B): (E, A): (C, S): (D, S): []
 CLOSED (B, A): (A, S): (S, null): []

4.

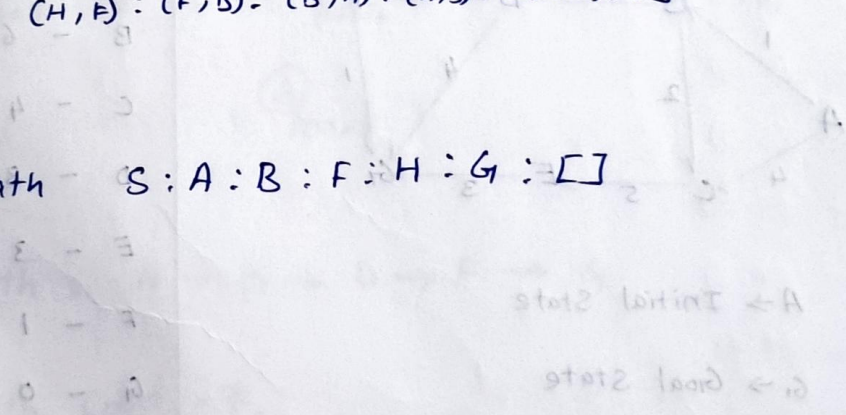
NODE F
 close (F, B)
 Move Gen B: E: H: K: []
 new nodes H: K: []
 new pairs (H, F): (K, F): []
 OPEN (H, F): (K, F): (E, A): (C, S): (D, S): []
 CLOSED (F, B): (B, A): (A, S): (S, null): []

5.

NODE H
 Close (H, F)
 move Gen C: D: F: G: []
 new nodes G: []
 new Pairs (G, H): []
 OPEN (G, H): (K, F): (E, A): (C, S): (D, S): []
 CLOSED (H, F): (F, B): (B, A): (A, S): (S, null): []

6.

NODE G
 Goal G
 OPEN (G, H): (K, F): (E, A): (C, S): (D, S): []
 CLOSED (H, F): (F, B): (B, A): (A, S): (S, null): []
 Path S: A: B: F: H: G: []



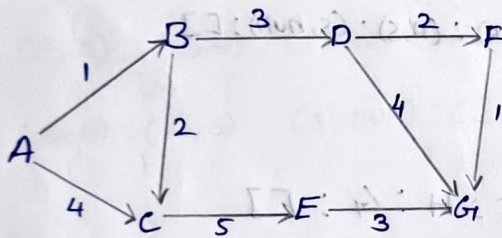
A* Algorithm

- Cost function $f(n) = g(n) + h(n)$
- $f(n)$ is the estimated total path to reach goal
- $g(n)$ - cost to reach the node
- $h(n)$ - heuristic value
- A* algorithm used by the computer for search purpose

$$f(n) = g(n) + h(n)$$

$g(n)$ = Cost from initial state to the state at the current node n .

$h(n)$ = estimated cost from the start at node n to goal state



A → Initial State

G → Goal State

A - 5

B - 6

C - 4

D - 3

E - 3

F - 1

G - 0

$$A \rightarrow B$$

$$f(B) = g(B) + h(B) = 1 + 6 = 7$$

$$A \rightarrow C$$

$$f(C) = g(C) + h(C) = 4 + 4 = 8$$

$$A \rightarrow B \rightarrow D$$

$$f(D) = 4 + 3 = 7$$

$$A \rightarrow B \rightarrow C$$

$$f(C) = 3 + 4 = 7$$

$$A \rightarrow B \rightarrow C \rightarrow E$$

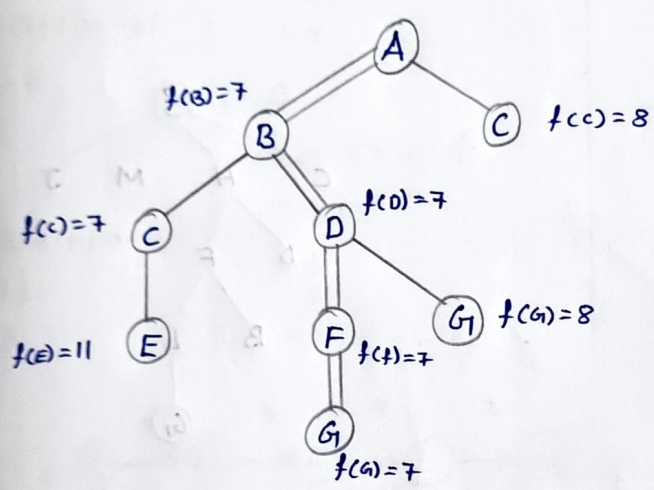
$$f(E) = 8 + 3 = 11$$

$$A \rightarrow B \rightarrow D \rightarrow G$$

$$f(G) = 8 + 0 = 8$$

$$A \rightarrow B \rightarrow D \rightarrow F \rightarrow G$$

$$f(G) = 7 + 0 = 7$$



Path : $A \rightarrow B \rightarrow D \rightarrow F \rightarrow G$

Path Cost = 7.

- (1,0) - A
- (1,1) - B
- (2,0) - C
- (2,2) - D
- (2,1) - E
- (4,1) - F
- (3,0) - G
- (F,1) - H
- (3,1) - I
- (1,2) - J
- (2,1) - K
- (3,1) - L
- (F,1) - M
- (0,3) - N
- (0,0) - O
- (3,0) - P
- (1,1) - Q
- (2,1) - R
- (0,1) - S

Manhattan Distance

$$D = |x_1 - x_2| + |y_1 - y_2|$$

Euclidean Distance

$$D = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Manhattan distance

A - (0, 3)

B - (1, 1)

C - (1, 8)

D - (2, 5)

E - (2, 15)

F - (4, 4)

G - (9, 3)

H - (4, 7)

I - (4, 13)

J - (6, 11)

K - (7, 15)

L - (7, 3)

M - (7, 7)

N - (8, 0)

O - (9, 9)

P - (10, 13)

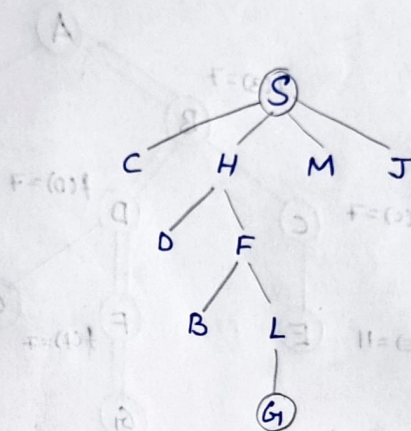
Q - (11, 1)

R - (11, 5)

S - (4, 10)

⑤ - Starting

⑥ - Destination



Path: A → B → D → F → L → G

Path Cost: F

$$D = |x_1 - x_2| + |y_1 - y_2|$$

(S, C)

$$\begin{aligned} D_1 &= |4-1| + |10-8| \\ &= 3+2 \\ &= 5 \end{aligned}$$

(S, H)

$$\begin{aligned} D_2 &= |4-4| + |10-7| \\ &= 0+3 \\ &= 3 \end{aligned}$$

(S, M)

$$\begin{aligned} D_3 &= |4-7| + |10-7| \\ &= 3+3 \\ &= 6 \end{aligned}$$

(S, J)

$$\begin{aligned} D_4 &= |4-8| + |10-11| \\ &= 2+1 \\ &= 3 \end{aligned}$$

(H, D)

$$\begin{aligned} D_5 &= |4-2| + |7-5| \\ &= 2+2 \\ &= 4 \end{aligned}$$

(H, F)

$$\begin{aligned} D_6 &= |4-4| + |7-4| \\ &= 0+3 \\ &= 3 \end{aligned}$$

(F, L)

$$\begin{aligned} D_7 &= |4-7| + |4-3| \\ &= 3+1 \\ &= 4 \end{aligned}$$

(F, B)

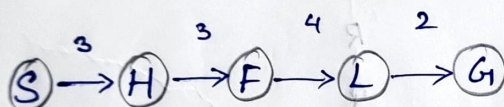
$$\begin{aligned} D_8 &= |4-1| + |4-1| \\ &= 3+3 \\ &= 6 \end{aligned}$$

(L, N)

$$\begin{aligned} D_9 &= |7-8| + |3-0| \\ &= 1+3 \\ &= 4 \end{aligned}$$

(L, G)

$$\begin{aligned} D_{10} &= |7-9| + |3-3| \\ &= 2+0 \\ &= 2 \end{aligned}$$



Total distance:

$$3 + 3 + 4 + 2 = 12$$